# ImageDescriptors

Informe técnico de arquitectura, modelos, parámetros y flujo experimental

Documento generado a partir del estado del repositorio

2 de noviembre de 2026

**Resumen**

ImageDescriptors es un sistema de visión por computador para describir y recuperar vehículos en imágenes de vigilancia. El proyecto combina el dataset VeRi, composición sintética sobre fondos de calles, detección YOLO, clasificación multitarea supervisada, modelos visión-lenguaje, embeddings locales, bases SQLite y una aplicación web. Este documento reconstruye la arquitectura implementada, especifica los parámetros que controlan cada etapa, describe los formatos de datos y señala las condiciones que pueden cambiar el resultado experimental o impedir una ejecución reproducible.

## Índice

**1. Alcance y estado del documento** **2**

**2. Resumen de la arquitectura** **2**
2.1. Flujo completo . . . 2
2.2. Estructura del repositorio . . . 2

**3. Datos y preparación** **3**
3.1. VeRi . . . 3
3.2. Clases y vocabularios . . . 3

**4. Generación del dataset sintético** **4**
4.1. Implementación . . . 4
4.2. Splits y parámetros . . . 4
4.3. Factores de validez . . . 4

**5. Detección con YOLO** **5**
5.1. Entrenamiento . . . 5
5.2. Inferencia y métricas . . . 5

**6. Clasificación multitarea supervisada** **5**
6.1. Arquitectura . . . 5
6.2. Datos, entrenamiento y ponderación . . . 6
6.3. Evaluación . . . 6

**7. Generación de descriptores multimodales** **6**
7.1. Florence-2 . . . 6
7.2. Qwen2-VL . . . 6

**8. Bases de datos** **7**
8.1. Atributos fijos . . . 7
8.2. Descripciones y embeddings . . . 7

**9. LLM local y recuperación** **8**
9.1. Configuración Ollama . . . . . 8
9.2. Modos del servicio . . . . . 8

**10. Aplicación web y API** **8**
10.1. Endpoints . . . . . 8

**11. Ejecución reproducible** **9**
11.1. Preparación . . . . . 9
11.2. Orden recomendado . . . . . 9

**12. Parámetros que más influyen en el resultado** **9**

**13. Limitaciones y observaciones técnicas** **10**

**14. Artefactos y reproducibilidad** **10**

**15. Conclusión** **11**

# 1 Alcance y estado del documento

Este informe describe el código presente en el repositorio, no una arquitectura idealizada. Los nombres de modelos, hiperparámetros, rutas, formatos y valores por defecto se derivan de los scripts de `src/`. La carpeta que contiene este informe está excluida de Git a petición del proyecto; por tanto, el PDF y los auxiliares de compilación son artefactos locales.

El proyecto tiene cuatro objetivos técnicos relacionados:

1. generar datos de detección realistas a partir de VeRi;
2. localizar vehículos y extraer regiones de interés;
3. predecir atributos visuales (color y tipo) con modelos generativos o supervisados;
4. consultar las descripciones con filtros estructurados, similitud semántica y una combinación híbrida.

# 2 Resumen de la arquitectura

## 2.1 Flujo completo

La arquitectura implementada se puede resumir así:

Listing 1: Flujo lógico del sistema

```
VeRi + streetscapes
       |
       v
Dataset sintetico: imagen 640x640 + bbox YOLO + metadata
       |
       +--> YOLO: deteccion de vehiculos --> bbox / ROI
       |
       +--> Florence-2 o Qwen2-VL --> descripcion y atributos
                                      |
                                      v
                        florence_results.json / CSV
                                      |
                                      v
              Ollama + nomic-embed-text-v2-moe --> SQLite de embeddings
                                      |
                                      v
Consulta en espanol --> parser LLM + reglas --> fixed / semantic / hybrid
                                      |
                                      v
                          API HTTP y frontend web
```

En paralelo, el extractor supervisado consume las etiquetas XML de VeRi y entrena una red multitarea con dos cabezas: color y tipo de vehículo. Sus checkpoints y métricas se usan como referencia experimental, no como dependencia obligatoria de la búsqueda web.

## 2.2 Estructura del repositorio

| Ruta | Responsabilidad |
|---|---|
| `VeRi/` | Imágenes, XML y archivos auxiliares del dataset real. |
| `streetscapes/` | Fondos de calles usados en la composición sintética. |
| `synthetic_dataset/` | Imágenes y etiquetas YOLO generadas; contenido ignorado por Git. |

| Ruta | Responsabilidad |
|---|---|
| `src/dataset/` | Composición, letterbox, cajas y generación de splits. |
| `src/Yolo/` | Entrenamiento, inferencia, ROI y métricas de detección. |
| `src/feature_generator/` | Inferencia Florence-2/Qwen-VL y normalización de respuestas. |
| `src/feature_extractor/` | Red multitarea, dataloaders, entrenamiento y evaluación. |
| `src/database/fixed_features/` | CSV timestamped, base de atributos y acceso SQL. |
| `src/database/generated_features/` | Conversión de descripciones a embeddings SQLite. |
| `src/local_llm/` | Cliente Ollama, prompts y embeddings normalizados. |
| `src/retrieval/` | Parser, reglas, fuentes y servicio de búsqueda. |
| `web_app.py`, `web/` | Servidor HTTP, API y frontend local. |

# 3 Datos y preparación

## 3.1 VeRi

El código utiliza principalmente:

- `image_train/`: imágenes para entrenamiento supervisado y muestreo de train/validation sintético;
- `image_test/`: imágenes de test y fuente de la base fija;
- `image_query/`: consultas originales de VeRi;
- `train_label.xml` y `test_label.xml`: atributos `imageName`, `colorID` y `typeID`.

La documentación original de VeRi impone condiciones de uso y no redistribución comercial. El repositorio mantiene el dataset fuera de Git mediante `.gitignore`.

## 3.2 Clases y vocabularios

El sistema trabaja con diez colores y nueve tipos. El orden es significativo porque coincide con los índices de las cabezas de clasificación y con los catálogos de recuperación:

Cuadro 2: Catálogos utilizados por clasificación y recuperación.

| ID | Color | ID | Tipo |
|---|---|---|---|
| 0 | yellow | 0 | sedan |
| 1 | orange | 1 | suv |
| 2 | green | 2 | van |
| 3 | gray | 3 | hatchback |
| 4 | red | 4 | mpv |
| 5 | blue | 5 | pickup |
| 6 | white | 6 | bus |
| 7 | golden | 7 | truck |
| 8 | brown | 8 | estate |
| 9 | black | | |

Los aliases en español incluyen, entre otros, “rojo” → `red`, “gris/plata” → `gray`, “camioneta” → `suv,pickup` y “camión” → `truck`. Un alias ambiguo produce varios tipos, lo que modifica el conjunto de candidatos del modo híbrido.

# 4 Generación del dataset sintético

## 4.1 Implementación

`src/dataset/create_synthetic.py` compone un vehículo sobre un fondo de calle mediante OpenCV. El algoritmo:

1. lee ambas imágenes en BGR;
2. redimensiona el fondo manteniendo proporción y limitando su dimensión mayor a 640 píxeles;
3. elige un área objetivo aleatoria entre 15 % y 30 % del fondo;
4. calcula una escala cuadrática a partir del área del recorte;
5. limita la escala para que el vehículo quepa completamente;
6. selecciona una posición aleatoria válida;
7. degrada dinámicamente el fondo según la escala;
8. mezcla el recorte mediante una máscara con gradiente de borde;
9. aplica letterbox a 640×640;
10. transforma la caja a formato YOLO normalizado.

Para una caja $(x, y, w, h)$ en la imagen final de tamaño 640, la etiqueta se calcula como:

$$x_c = \frac{x + w/2}{640}, \qquad y_c = \frac{y + h/2}{640}, \qquad w_n = \frac{w}{640}, \qquad h_n = \frac{h}{640}. \tag{1}$$

La clase es siempre 0, `vehicle`.

## 4.2 Splits y parámetros

`make_dataset.py` usa semilla 42, toma hasta 5000 imágenes para train y 1000 para validation desde `image_train`, y conserva el conjunto de `image_test` como test independiente. Los fondos se recorren cíclicamente. Cada split genera imágenes, archivos `.txt` y un CSV de metadata con la imagen original, el fondo, color/tipo y la caja.

Cuadro 3: Parámetros que afectan el dataset sintético.

| **Parámetro** | **Valor** | **Efecto** |
|---|---|---|
| `CANTIDAD_TRAIN` | 5000 | Tamaño máximo de train. |
| `CANTIDAD_VAL` | 1000 | Tamaño máximo de validation. |
| `AREA_RANGE` | [0.15, 0.30] | Fracción del área del fondo ocupada por el vehículo. |
| `SEED` | 42 | Reproducibilidad de muestreo y posiciones. |
| `LOG_EVERY` | 500 | Frecuencia de mensajes de progreso. |
| `letterbox` | 640×640 | Tamaño requerido por el entrenamiento YOLO. |

## 4.3 Factores de validez

La composición sintética introduce un cambio de dominio: el vehículo real de VeRi aparece sobre un fondo diferente y con una escala artificial. El rango de área, el blending, el desenfoque/degradación, la distribución de fondos y la selección aleatoria determinan la dificultad de YOLO. Un dataset sintético fácil puede producir métricas altas pero generalizar peor a imágenes reales.

# 5 Detección con YOLO

## 5.1 Entrenamiento

`src/Yolo/finetuning.py` carga `yolo26s.pt` y utiliza Ultralytics para fine-tuning. El archivo `synthetic.yaml` declara train, validation y test con una única clase. Los valores actuales son:

Cuadro 4: Configuración de fine-tuning YOLO.

| **Parámetro** | **Valor** |
|---|---|
| Modelo base | YOLO26s (`yolo26s.pt`) |
| Épocas | 50 |
| `imgsz` | 640 |
| `batch` | 16 |
| `device` | GPU 0 |
| `workers` | 6 |
| Clase | vehicle (ID 0) |

El modelo pequeño `yolo26n.pt` también está presente, pero el script de entrenamiento selecciona explícitamente la variante `s`. Las salidas se almacenan normalmente en `runs/detect/.../weights/best.pt`, carpeta excluida de Git.

## 5.2 Inferencia y métricas

`detectar_vehiculos` filtra por confianza, dibuja las cajas y serializa cada detección con clase, nombre, confianza y coordenadas absolutas. El umbral por defecto es 0.25. `extract_roi` toma la primera caja detectada y recorta la imagen original; no implementa selección por mayor confianza ni una ruta explícita para el caso sin detecciones.

`metrics.py` calcula precision, recall, mAP@50, mAP@50–95 y tiempos de preprocesamiento, inferencia y postprocesamiento. El FPS reportado es:

$$\mathrm{FPS} = \frac{1000}{t_{\mathrm{pre}} + t_{\mathrm{inf}} + t_{\mathrm{post}}}, \tag{2}$$

donde los tiempos están en milisegundos por imagen.

# 6 Clasificación multitarea supervisada

## 6.1 Arquitectura

`MultiTaskVehicleNet` usa un backbone de `torchvision` y reemplaza su clasificador por dos cabezas lineales:

$$z_{\mathrm{color}} = W_c h + b_c, \qquad z_{\mathrm{type}} = W_t h + b_t, \tag{3}$$

donde $h$ es la representación compartida. Los backbones soportados son:

- `efficientnet_b0`;
- `mobilenet_v3_small`.

Cada cabeza produce 10 y 9 logits respectivamente. El argumento `pretrained=True` solicita pesos preentrenados de `torchvision` cuando la arquitectura los soporta.

### 6.2 Datos, entrenamiento y ponderación

El dataloader lee VeRi y construye lotes de imágenes con etiquetas de color y tipo. El entrenamiento usa Adam y dos pérdidas Cross Entropy ponderadas:

$$\mathcal{L} = \mathcal{L}_{color}(z_c, y_c; w_c) + \mathcal{L}_{type}(z_t, y_t; w_t). \tag{4}$$

Los pesos de clase reducen el efecto del desbalance. En particular, el vector de pesos de tipo contiene un peso muy alto para `estate` (34.95), por lo que errores de esa clase influyen fuertemente en la pérdida.

Cuadro 5: Parámetros de entrenamiento observados.

| **Parámetro** | **Valor** | **Observación** |
|---|---|---|
| Backbones ejecutados en `__main__` | MobileNetV3-Small y EfficientNet-B0 | 15 épocas cada uno. |
| Batch | 32 | Afecta memoria y estabilidad del gradiente. |
| Learning rate | 0.0003 | Valor usado en la ejecución principal. |
| Optimizador | Adam | Actualiza todos los parámetros. |
| Checkpoints | `weights/best_{backbone}.pth` | Se guarda la menor pérdida de validation. |
| Métrica de selección | `avg_val_loss` | No selecciona directamente por accuracy. |

La función puede invocarse con otros valores; sus valores por defecto son 10 épocas, batch 32, learning rate 0.001 y EfficientNet-B0, pero el bloque ejecutable del archivo usa 15 épocas y 0.0003 para ambos backbones. Esta diferencia debe considerarse al reproducir experimentos.

### 6.3 Evaluación

`evaluation.py` genera accuracy, F1 macro, reportes por clase y matrices de confusión para color y tipo. También mide tiempo por imagen y FPS. Las clases ausentes del test se excluyen del F1 macro para evitar penalización por categorías sin muestras.

## 7 Generación de descriptores multimodales

### 7.1 Florence-2

`run_florence.py` carga `microsoft/Florence-2-base` con `AutoProcessor` y `AutoModelForCausalLM`. Selecciona CUDA si está disponible y CPU en caso contrario. El benchmark usa la tarea `<MORE_DETAILED_CAPTION>`; la salida cruda se guarda en JSON y luego se normaliza para evaluación.

### 7.2 Qwen2-VL

El benchmark común llama `load_light_qwen_model`, que carga `Qwen/Qwen2-VL-2B-Instruct` en 4 bits mediante `bitsandbytes`, `device_map=.ªuto"` y `float16`. El script conserva además una función opcional para `Qwen/Qwen2-VL-7B-Instruct` en 8 bits. La versión ligera es la que usa `generate_predictions.py`.

El prompt de Qwen solicita JSON con atributos fijos (color y tipo) y dinámicos (hora, ángulo y accesorios). `parse_outputs.py` restringe color y tipo a los catálogos, corrige sinónimos como `silver`/`grey` hacia `gray` y produce CSV comparable entre modelos.

Cuadro 6: Entornos de inferencia generativa.

| Entorno | Paquetes/modelo relevante | Riesgo o condición |
|---|---|---|
| Florence | Python 3.10, Torch 2.6.0+cu124, Transformers 4.47.1, Accelerate 1.2.1, timm | Requiere descarga de Hugging Face y VRAM suficiente. |
| Qwen | Python 3.10, Torch 2.6.0+cu124, Transformers 4.51.3, qwen-vl-utils 0.0.14 | La cuantización ligera requiere bitsandbytes y GPU práctica. |

# 8 Bases de datos

## 8.1 Atributos fijos

`add_timestamps.py` lee `src/feature_extractor/data/test_features.csv` y asigna `captured_at` de forma determinista. Para cada imagen calcula

$$\text{offset} = \text{SHA256}(\texttt{capture-time-v1}, 42, \text{imageName}) \text{ mód duracion}, \tag{5}$$

y suma el offset al intervalo 2026-10-01 00:00:00–06:00 hasta 2026-11-01 00:00:00–06:00.

`build_database.py` crea `vehicles.sqlite3`, tabla `vehicles`, e índices para `colorID`, `typeID` y `captured_at`. La consulta de `repository.py` es parametrizada y omite los filtros cuyo valor es `None`.

## 8.2 Descripciones y embeddings

`build_embeddings.py` transforma `florence_results.json` en `florence.sqlite3`. La base contiene:

- `descriptions(image_id, text, text_sha256)`;
- `embeddings(image_id, variant, chunk_idx, text, vector)`;
- `meta(key, value)`.

Se generan las variantes `full` (un vector por imagen) y `sentences` (un vector por oración con al menos 15 caracteres). El modelo es `nomic-embed-text-v2-moe`, la dimensión esperada es 768, el batch es 16 y cada bloque de hasta 128 textos se confirma en SQLite. El vector se normaliza:

$$\hat{v} = \frac{v}{\|v\|_2}. \tag{6}$$

La similitud coseno entre consulta y documento se reduce a $\hat{q}^T \hat{v}$.

El proceso es incremental: compara SHA-256, borra embeddings de descripciones modificadas o ausentes, y procesa únicamente pendientes. Si un lote falla, `safe_embed` reintenta cada texto por separado. `REBUILD=True` borra las tablas antes de reconstruir y es obligatorio si cambia el modelo o la dimensión.

# 9 LLM local y recuperación

## 9.1 Configuración Ollama

Cuadro 7: Configuración central en `src/local_llm/config.py`.

| Parámetro | Valor |
|---|---|
| OLLAMA_ENDPOINT | http://127.0.0.1:11434 |
| CHAT_MODEL | qwen2.5:3b |
| EMBED_MODEL | nomic-embed-text-v2-moe |
| EMBED_DIM | 768 |
| CHAT_CONTEXT | 4096 |
| TEMPERATURE | 0.0 |
| TIMEOUT | 120 s |
| EMBED_BATCH_SIZE | 16 |

`client.py` exige JSON para el modelo de chat. El prompt declara colores, tipos, aliases, periodos y el esquema: `types`, `colors`, exclusiones, intervalos horarios, fechas, `semantic_text`, términos no soportados y límite. `query_parser.py` valida y registra warnings; `rules.py` aplica correcciones deterministas a negaciones, aliases ambiguos y periodos.

## 9.2 Modos del servicio

**`fixed`** El chat produce filtros; se consulta SQLite con atributos fijos. Si no hay filtros se devuelve la base sin restricción y se registra un warning.

**`semantic`** La consulta completa se embebe sin usar el modelo de chat y se ordena contra `florence.sqlite3`.

**`hybrid`** Los filtros reducen candidatos y el texto residual los ordena semánticamente. Si no hay texto residual, funciona como fixed. La constante `HYBRID_USE_FULL_QUERY=False` controla esta decisión.

Los límites son 20 por defecto y 100 como máximo. En semantic/hybrid, `SearchResult.total` es el número de imágenes consideradas para el ranking, no necesariamente el número mostrado.

# 10 Aplicación web y API

`web_app.py` usa `ThreadingHTTPServer` de la biblioteca estándar. Escucha en `127.0.0.1:8765`, sirve `web/index.html`, busca imágenes en `VeRi/image_test` y limita resultados a 100. Al iniciar intenta construir las fuentes fixed y generated; si faltan bases o dependencias, conserva el error por modo y la aplicación puede arrancar parcialmente.

## 10.1 Endpoints

| Endpoint | Comportamiento |
|---|---|
| `GET /` | Sirve el frontend HTML. |
| `GET /api/status` | Devuelve modos disponibles y errores de inicio. |
| `POST /api/search` | Recibe `query`, `mode` y `limit`; devuelve resultados. |
| `GET /images/<id>` | Sirve imágenes solo dentro de `VeRi/image_test`. |

La API devuelve errores 400 para JSON o argumentos inválidos, 503 cuando el modo no tiene dependencias disponibles y 500 para errores internos. La resolución de rutas comprueba que la imagen permanezca dentro de la carpeta permitida, evitando atravesar directorios mediante el identificador de imagen.

# 11 Ejecución reproducible

## 11.1 Preparación

```
# Entornos de los modelos vision-lenguaje
conda env create -f env_florence.yml
conda env create -f env_qwen.yml

# Modelos locales de Ollama
ollama serve
ollama pull qwen2.5:3b
ollama pull nomic-embed-text-v2-moe
```

Los scripts contienen algunas rutas absolutas heredadas de una instalación anterior, especialmente `make_dataset.py`, `synthetic.yaml`, `metrics.py` y bloques `main`. Es necesario sustituirlas por la raíz actual antes de ejecutar. Los comandos de los scripts de feature generator deben ejecutarse desde `src/feature_generator` por sus imports locales.

## 11.2 Orden recomendado

```
# Desde la raiz
python -m src.dataset.make_dataset

cd src/Yolo
python finetuning.py
python metrics.py
cd ../..

python src/database/fixed_features/add_timestamps.py
python src/database/fixed_features/build_database.py
python src/database/generated_features/build_embeddings.py

python web_app.py
```

Para obtener las descripciones primero se ejecuta el benchmark de Florence o Qwen y se generan los JSON/CSV en `src/feature_generator/data/`. Después se construye la base de embeddings y finalmente se inicia la web.

# 12 Parámetros que más influyen en el resultado

1. **Distribución sintética:** área 15–30 %, posición, escala, fondos, letterbox y semilla cambian la apariencia de los ejemplos de YOLO.
2. **Backbone y pesos iniciales:** EfficientNet-B0 y MobileNetV3-Small tienen capacidad, latencia y sesgo distintos.
3. **Desbalance:** los pesos de Cross Entropy, especialmente el peso 34.95 de `estate`, cambian la frontera de decisión.
4. **Modelo multimodal y prompt:** Florence genera caption detallado, mientras Qwen recibe un esquema de atributos; sus salidas no son directamente equivalentes sin `parse_outputs.py`.

5. **Cuantización:** Qwen-2B en 4 bits reduce VRAM pero puede cambiar la calidad frente a 7B en 8 bits.
6. **Embedding:** modelo, dimensión, prefijos, variante `full/sentences` y normalización determinan el ranking.
7. **Parser y reglas:** aliases, negaciones y periodos pueden ampliar o reducir los candidatos antes de la similitud.
8. **Corte de recuperación:** límite 20/100, variante SQLite y `HYBRID_USE_FULL_QUERY` afectan los resultados visibles.
9. **Rutas y artefactos:** pesos, bases, XML e imágenes no se versionan; una ejecución sin esos artefactos no representa el sistema completo.

## 13 Limitaciones y observaciones técnicas

- No existe un archivo único de dependencias para YOLO, dataset y retrieval; los entornos Florence y Qwen no son equivalentes.
- `synthetic.yaml` y varios bloques de prueba contienen rutas absolutas. Esto reduce la portabilidad.
- `extract_roi` asume al menos una detección y usa la primera caja, no necesariamente la de mayor confianza.
- La fuente generated carga todos los vectores en memoria y hace búsqueda exacta con NumPy; el coste crece linealmente con el número de embeddings.
- La generación de embeddings reintenta fallos, pero los deja para una ejecución posterior y solo imprime el resumen.
- La búsqueda semántica depende de descripciones generadas; errores de Florence o Qwen se propagan a la recuperación.
- El modo fixed depende del modelo de chat local para interpretar lenguaje natural, aunque la consulta SQL sea parametrizada.
- La web es un servidor local de desarrollo, no un servicio autenticado para exposición pública.

## 14 Artefactos y reproducibilidad

Los siguientes artefactos se ignoran deliberadamente: `*.pt`, `*.pth`, datasets, `runs/`, CSV/figuras de features y bases generadas. Para reproducir resultados se debe conservar, como mínimo:

1. la revisión del código;
2. las versiones de Python, PyTorch, Transformers y Ultralytics;
3. los modelos descargados y sus variantes/cuantización;
4. los datasets VeRi y streetscapes;
5. los parámetros de generación y entrenamiento;
6. los JSON/CSV de descripciones;
7. la configuración de Ollama y la base de embeddings.

# 15 Conclusión

ImageDescriptors es una cadena modular que comienza con datos de vehículos y termina en una interfaz de búsqueda multimodal. La localización YOLO, la clasificación multitarea y los modelos visión-lenguaje pueden evaluarse por separado; la capa de bases y recuperación los integra mediante atributos estructurados y descripciones vectorizadas. El resultado final depende tanto de los modelos como de la preparación sintética, los vocabularios, los prompts, la cuantización, la configuración de Ollama y los artefactos persistidos. Para comparar experimentos es esencial registrar todos esos elementos y corregir las rutas absolutas antes de ejecutar el pipeline en otra máquina.